



COS20028 – BIG DATA ARCHITECTURE AND APPLICATION

Dr Pei-Wei Tsai (Lecturer, Tutor, and Unit Convenor)
ptsai@swin.edu.au, EN508d





Week 8



**Hadoop Tools
for Data
Acquisition**

Apache Kafka

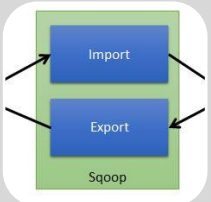




HADOOP TOOLS FOR DATA ACQUISITION

Data Acquisition with Hadoop

We are about to learn:



How to load data from an existing RDBMS into HDFS using Sqoop.



How to manage real-time data such as log files using Flume.



LOADING DATA INTO HDFS FROM AN RDBMS USING SQOOP

Importing Data from an RDBMS to HDFS

Typical scenario

- Data stored in an RDBMS is needed in a MapReduce job
 - Lookup tables
 - Legacy data

Possible to read directly from an RDBMS in your Mapper (Not recommended)

- Can lead to the equivalent of a distributed denial of service (DDoS) attack on your RDBMS.
- In practice: Don't do it!

Better idea: use Sqoop to import the data into HDFS beforehand.



Sqoop – SQL to Hadoop

Sqoop is an open source tool originally written at Cloudera.

- It is a tool designed for efficiently transferring bulk data between Apache Hadoop and structured datastores such as relational databases.
- Graduated from the incubator in March of 2012 and was once a Top-Level Apache Project.
- Unfortunately, it is just moved into the Apache Attic in June, 2021, because the community is lack of active Project Management Committee (PMC).

Imports tables from an RDBMS into HDFS

- Just one table.
- All tables in a database.
- Just portions of a table. (Sqoop supports a *WHERE* clause)

Sqoop – SQL to Hadoop

Uses MapReduce to actually import the data.

- “Throttles” the number of Mappers to avoid DDoS scenarios.
- Uses four Mappers by default.
- Value is configurable.

Uses a JDBC interface

- Should work with virtually any JDBC-compatible database.

Sqoop – SQL to Hadoop

Imports data to HDFS as delimited text files or SequenceFiles

- Default is a comma-delimited text file.

Can be used for incremental data imports

- First retrieves all rows in a table.
- Subsequently retrieves just rows created since the last import.

Generates a class file which can encapsulate a row of the imported data

- Useful for serialising and deserialising data in subsequent MapReduce jobs.

Custom Sqoop Connectors

Cloudera has partnered with other organisations to create custom Sqoop connectors

- Use a database's native protocols rather than JDBC.
- Provides much faster performance.

Current systems supported by custom connectors include:

- Netezza (by IBM: The next generation of Netezza is a part of IBM Cloud Pak for Data System.)
- Teradata (by Teradata: A cloud and data solution provider company.)
- Oracle Database (connector developed with Quest Software)

Custom connectors are not open source, but are free

- Available from the Cloudera Website.

Sqoop Basic Syntax

Standard Syntax

`sqoop tool-name [tool-options]`

Tools Include

`import`
`import-all-tables`
`list-tables`

Options include

`--connect`
`--username`
`--password`

Sqoop - Example

Example: Import a table called *employees* from a database called *personnel* in a MySQL RDBMS

```
sqoop import --username u05 --password u05passwd --connect  
jdbc:mysql://database.example.com/personnel --table employees
```

Example: As above, but only records with an ID greater than 1000

```
sqoop import --username u05 --password u05passwd --connect  
jdbc:mysql://database.example.com/personnel --table employees --  
where "id > 1000"
```

Importing an Entire Database with Sqoop

**Import all tables from the database
(fields will be tab-delimited)**

```
sqoop import-all-tables --connect  
jdbc:mysql://localhost/company --username  
u01 --password bigsecret --fields-  
terminated-by '\t' --warehouse-dir /mydata
```

Incremental Imports with Sqoop

What if new records are added to the database?

- Could re-import all records, but this is inefficient.

Sqoop's incremental append mode imports **ONLY** *new* records.
(Based on value of last record in specified column.)

```
sqoop import --connect jdbc:mysql://localhost/company --username u02 --  
password youllneverknow --warehouse-dir /mydata --table orders --  
incremental append --check-column order_id --last-value 13524
```


Sqoop – Other Options

Sqoop can take data from HDFS and insert it into an already-existed table in an RDBMS with the command

`sqoop export [options]`

For general Sqoop help:

`sqoop help`

For help on a particular command:

`sqoop help command`



MANAGING REAL-TIME DATA USING FLUME

Flume - Basics

Flume is a distributed, reliable, available service for efficiently moving large amounts of data as it is produced.

- It is ideally suited to gather logs from multiple systems and inserting them into HDFS as they are generated.

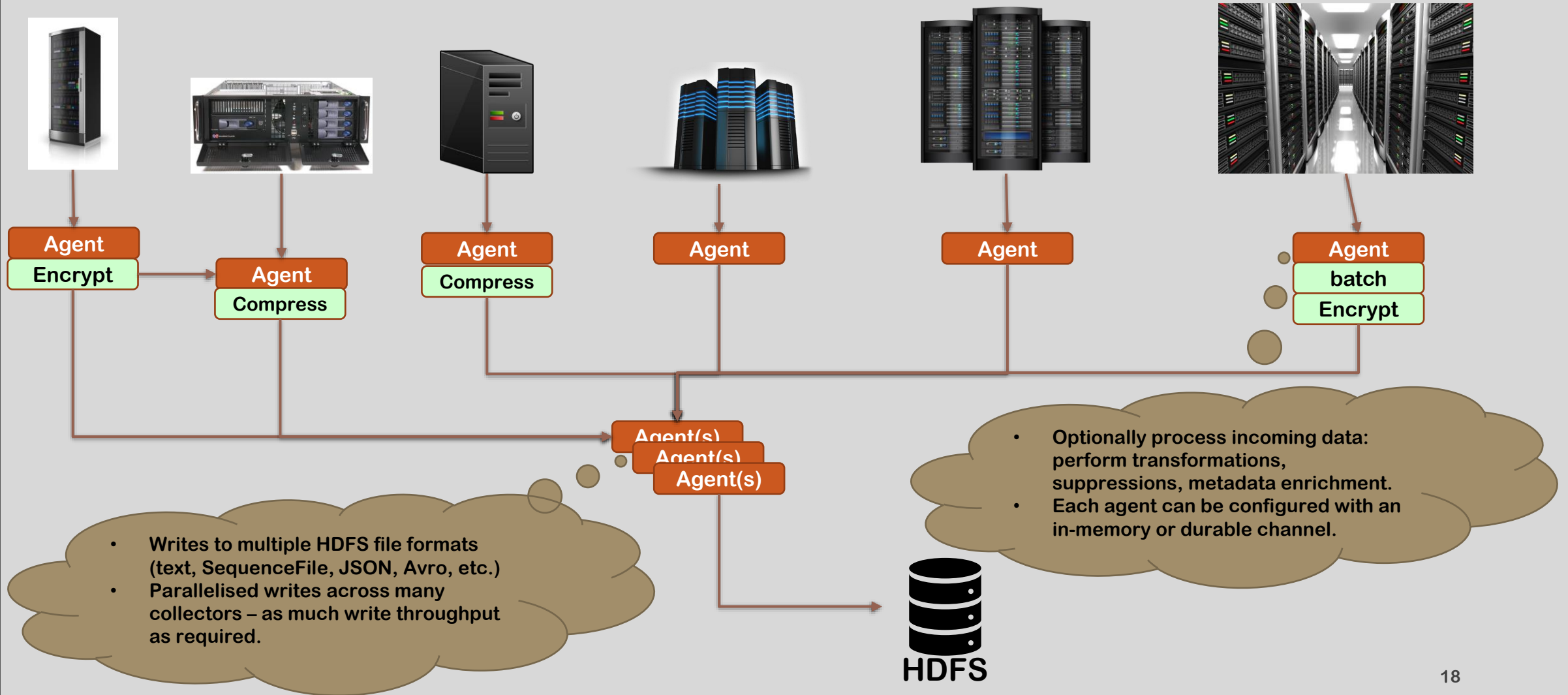
Flume is Open Source.

- Initially developed by Cloudera

Flume's design goals are:

- Reliability
- Scalability
- Extensibility

High-Level Overview of Flume



Flume – Agent Characteristics

Each Flume agent has a *source*, a *sink*, and a *channel*.

Source

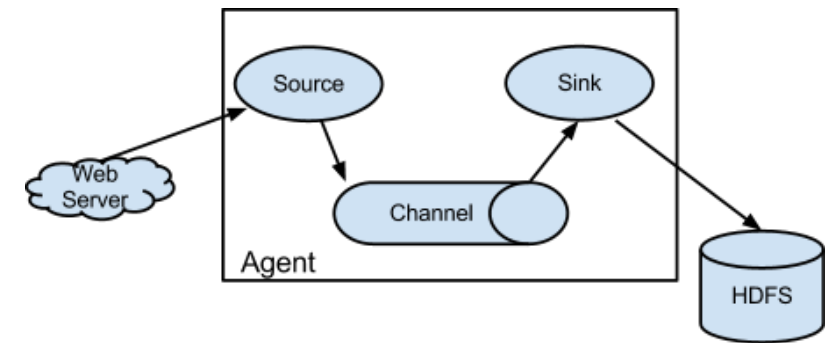
- Tells the node where to receive data from.

Sink

- Tells the node where to send data to.

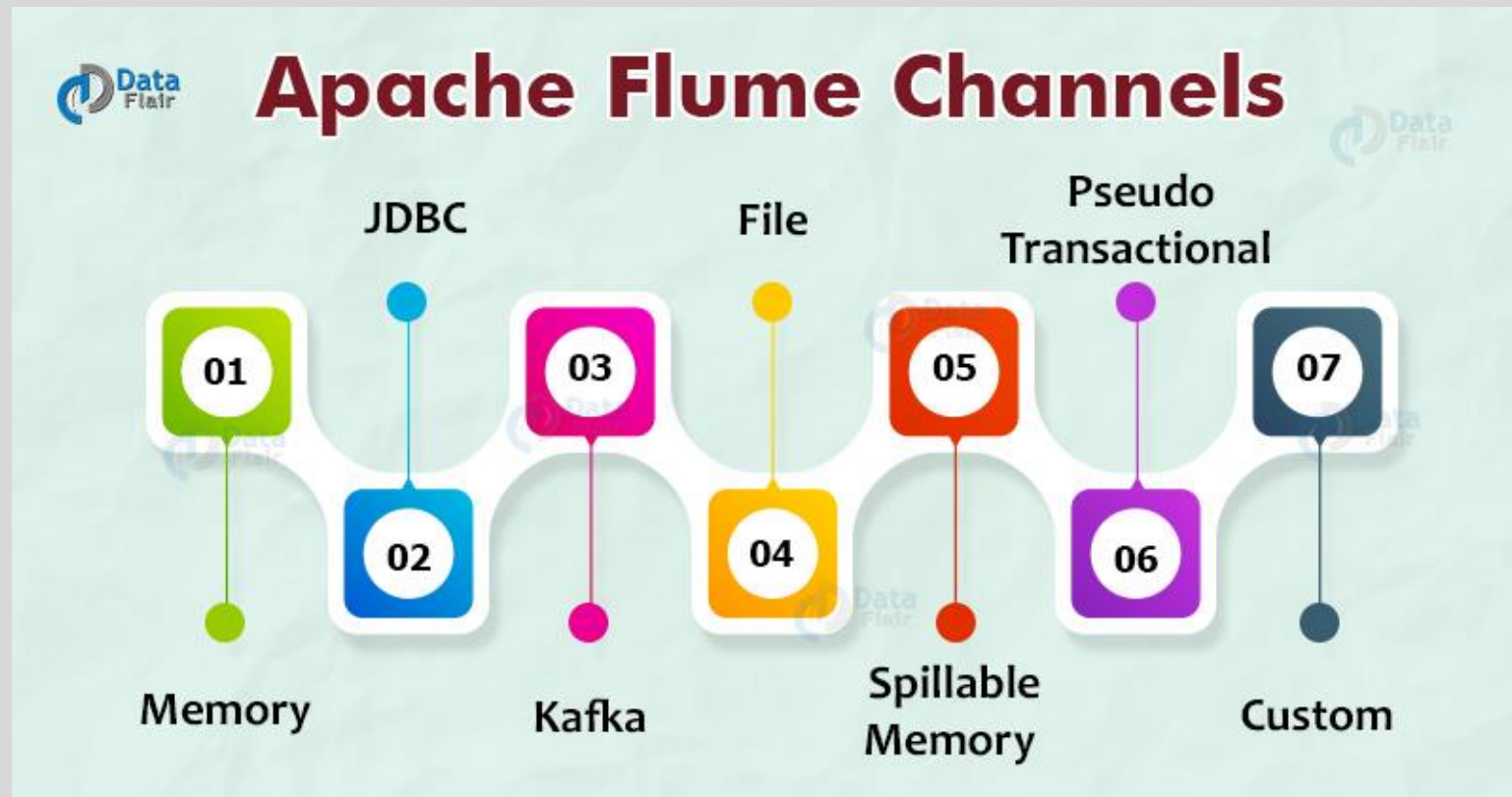
Channel

- A queue between the Source and Sink.
- Can be in-memory only or “Durable”.
 - Durable channels will not lose data if power is lost.



<https://flume.apache.org/>

Flume's Channels



Flume's Design Goals: Reliability

Channels provide Flume's reliability.

Memory Channel

- Data will be lost if power is lost.

File Channel

- Data store on disk.
- Guarantees durability of data in face of a power loss.

Data transfer between Agents and Channels is transactional.

- A failed data transfer to a downstream agent rolls back and retries.

Flume's Design Goals: Scalability

Can configure multiple Agents with the same task.

- Meaning, two Agents doing the job of one “collector” – if one agent fails then upstream agents would failover.

Scalability

- The ability to increase system performance linearly by adding more resources to the system.
- Flume scales horizontally.
 - As load increases, more machines can be added to the configuration.

Flume's Design Goals: Extensibility

Extensibility

- The ability to add new functionality to a system.

Flume can be extended by adding Sources and Sinks to existing storage layers or data platforms.

- General Sources include data from files, syslog, and standard output from a process.
- General Sinks include files on the local filesystem or HDFS.
- Developers can write their own Sources or Sinks.

Flume Usage Patterns

Flume is typically used to ingest log files from real-time systems such as Web servers, firewalls, and mail servers into HDFS.

Currently, it is in use in many large organisations, ingesting millions of events per day.

- At least one organisation is using Flume to ingest over 200 million events per day.

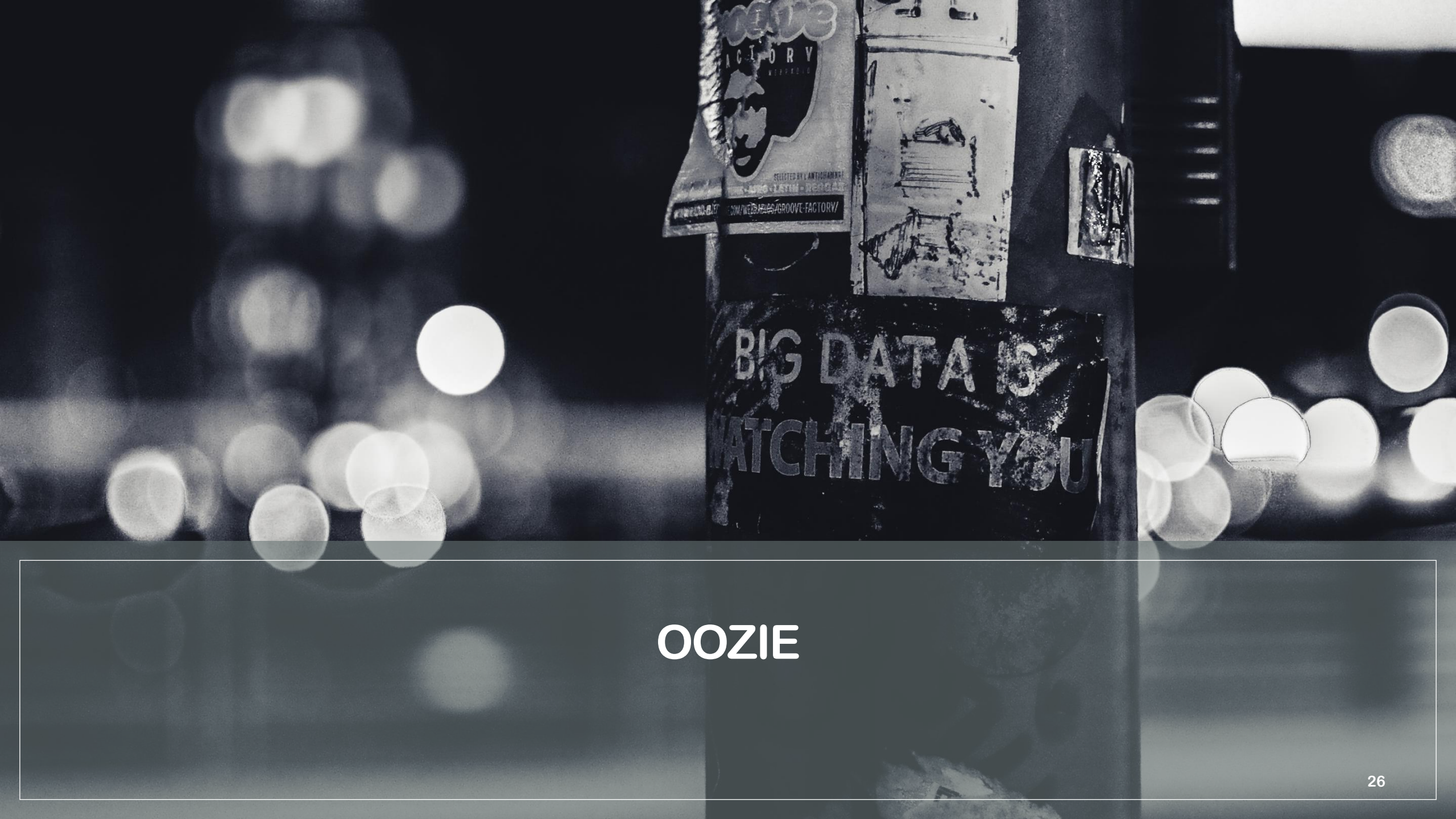
Flume is typically installed and configured by a system administrator.

- Check the Flume documentation if you intend to install it yourself.

Brief Summary

Sqoop is a tool to load data from a database into HDFS.

**Flume is a tool for managing real-time data.
(importing data from log files into HDFS)**



OOZIE

What is Oozie?

Apache Oozie

- It is a Job Scheduling System.
- Gets integrated with Hadoop Framework.
- Executes Multiple jobs in Parallel.
- It is an open source java web application.
- It uses Hadoop execution engine to execute the tasks.
- It is able to leverage the existing Hadoop machinery designed for many tasks.

An online short course introducing Apache Oozie can be found at:

- <https://www.youtube.com/watch?v=dEBnh8erPO8>

The Motivation for Oozie

Many problems cannot be solved with a single MapReduce job.

A *workflow* of jobs must be created if you are using Oozie.

A simple example workflow is given as follows:

- Run Job A
- Use output of Job A as input to Job B
- Use output of Job B as input to Job C
- Output of Job C is the final required output

Easy if the workflow is linear like this.

- Can be created as standard Driver code.

The Motivation for Oozie

If the workflow is more complex, Driver code becomes much more difficult to be maintained.

Example 1:

- **Running multiple jobs in parallel, using the output from all of those jobs as the input to the next job.**

Example 2:

- **Including Hive or Pig jobs as part of the workflow.**

Introduction to Oozie

Oozie is a “workflow engine”.

Runs on a server.

- Typically outside the cluster.

Runs workflows of Hadoop jobs.

- Including Pig, Hive, Sqoop jobs.
- Submits those jobs to the cluster based on a workflow definition.

Workflow definitions are submitted via HTTP.

Jobs can be run at specific times.

- One-off or recurring jobs are both supported.

Jobs can be run when data is present in a directory.

Oozie Workflow Basics

Oozie workflows are written in XML.

Workflow is a collection of actions.

- MapReduce jobs, Pig jobs, Hive jobs, etc.

A workflow consists of *control flow nodes* and *action nodes*.

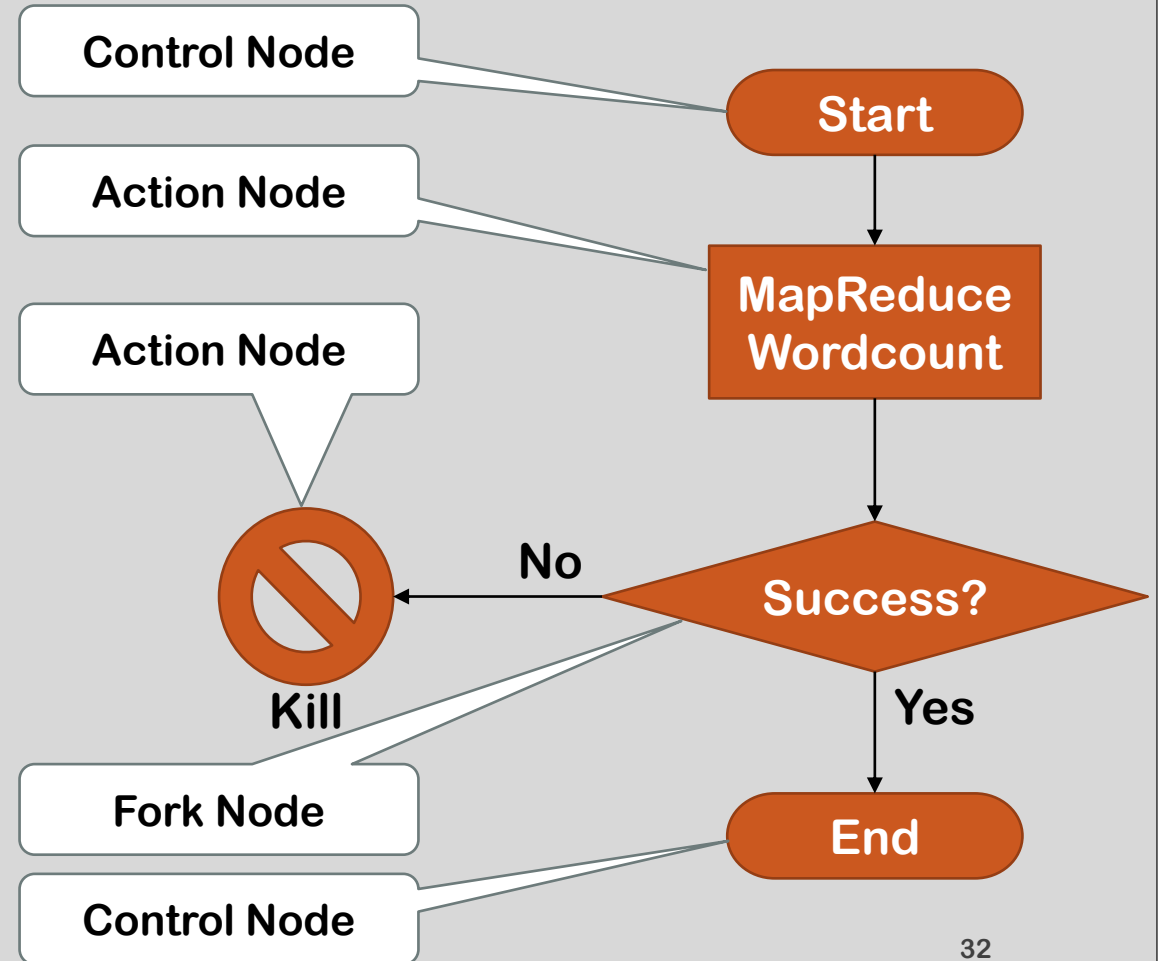
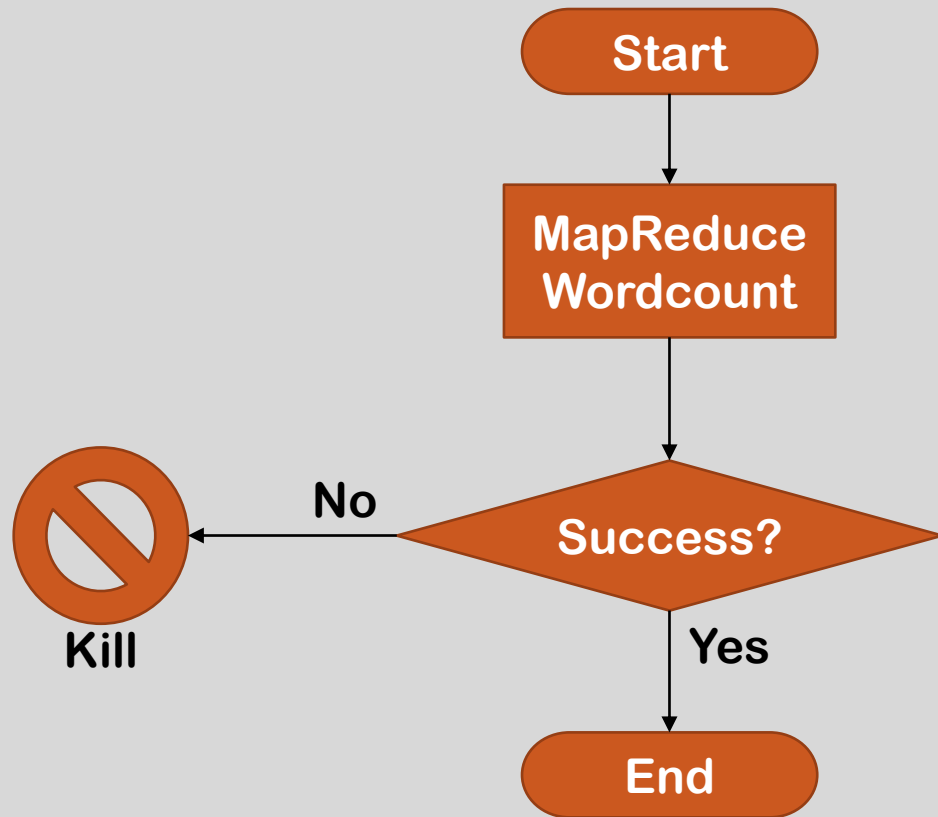
Control flow nodes define the beginning and the end of a workflow.

- They provide methods to determine the workflow execution path.
- Example: run multiple jobs simultaneously.

Action nodes trigger the execution of a processing task, such as:

- A MapReduce job.
- A Hive query.
- A Sqoop data import job.

Simple Oozie Example




```

<workflow-app name='wordcount-wf' xmlns="uri:oozie:workflow:0.1">
  <start to='wordcount' />
  <action name='wordcount'>
    <map-reduce>
      <job-tracker>${jobTracker}</job-tracker>
      <name-node>${nameNode}</name-node>
      <configuration>
        <property>
          <name>mapred.mapper.class</name>
          <value>org.myorg.WordCount.Map</value>
        </property>
        <property>
          <name>mapred.reducer.class</name>
          <value>org.myorg.WordCount.Reduce</value>
        </property>
        <property>
          <name>mapred.input.dir</name>
          <value>${inputDir}</value>
        </property>
        <property>
          <name>mapred.output.dir</name>
          <value>${outputDir}</value>
        </property>
      </configuration>
    </map-reduce>
    <ok to='end' />
    <error to='kill' />
  </action>
  <kill name='kill'>
    <message>Something went wrong: ${wf:errorCode('wordcount')}</message>
  </kill>
  <end name='end' />
</workflow-app>

```

SIMPLE OOZIE EXAMPLE

```
<workflow-app name='wordcount-wf' xmlns="uri:oozie:workflow:0.1">
```

A workflow is wrapped in the *workflow-app* entity.

```
  <property>
    <name>mapred.reducer.class</name>
    <value>org.myorg.WordCount.Reduce</value>
  </property>
  <property>
    <name>mapred.input.dir</name>
    <value>${inputDir}</value>
  </property>
  <property>
    <name>mapred.output.dir</name>
    <value>${outputDir}</value>
  </property>
</configuration>
</map-reduce>
<ok to='end' />
<error to='kill' />
</action>
<kill name='kill'>
  <message>Something went wrong: ${wf:errorCode('wordcount')}</message>
</kill>
<end name='end' />
</workflow-app>
```

SIMPLE OOZIE EXAMPLE

```
<workflow-app name='wordcount-wf' xmlns="uri:oozie:workflow:0.1">
```

```
<start to='wordcount' />
```

- The *start* node is the control node, which tells Oozie which workflow node should be run first.
- There must be one *start* node in an Oozie workflow.
- In our example, we are telling Oozie to start by transitioning to the *wordcount* workflow node.

```
    <property>
      <name>mapred.input.dir</name>
      <value>${inputDir}</value>
    </property>
    <property>
      <name>mapred.output.dir</name>
      <value>${outputDir}</value>
    </property>
  </configuration>
</map-reduce>
<ok to='end' />
<error to='kill' />
</action>
<kill name='kill'>
  <message>Something went wrong: ${wf:errorCode('wordcount')}</message>
</kill>
<end name='end' />
</workflow-app>
```

SIMPLE OOZIE EXAMPLE

```
<workflow-app name='wordcount-wf' xmlns="uri:oozie:workflow:0.1">
  <start to='wordcount' />
  <action name='wordcount'>
    <map-reduce>
```

- The *wordcount* action node defines a *map-reduce* action, which is a standard Java MapReduce job.

```
    </property>
    <property>
      <name>mapred.reducer.class</name>
      <value>org.myorg.WordCount.Reduce</value>
    </property>
    <property>
      <name>mapred.input.dir</name>
      <value>${inputDir}</value>
    </property>
    <property>
      <name>mapred.output.dir</name>
      <value>${outputDir}</value>
    </property>
  </configuration>
```

```
    </map-reduce>
    <ok to='end' />
    <error to='kill' />
  </action>
  <kill name='kill'>
    <message>Something went wrong: ${wf:errorCode('wordcount')}</message>
  </kill>
  <end name='end' />
</workflow-app>
```

SIMPLE OOZIE EXAMPLE

```
<workflow-app name='wordcount-wf' xmlns="uri:oozie:workflow:0.1">
  <start to='wordcount' />
  <action name='wordcount'>
```

```
    <map-reduce>
      <job-tracker>${jobTracker}</job-tracker>
      <name-node>${nameNode}</name-node>
      <configuration>
        <property>
          <name>mapred.mapper.class</name>
          <value>org.myorg.WordCount.Map</value>
        </property>
        <property>
          <name>mapred.reducer.class</name>
          <value>org.myorg.WordCount.Reduce</value>
        </property>
        <property>
          <name>mapred.input.dir</name>
          <value>${inputDir}</value>
        </property>
        <property>
          <name>mapred.output.dir</name>
          <value>${outputDir}</value>
        </property>
      </configuration>
    </map-reduce>
```

- Within the action, we define the job's properties.

```
    </kill>
  <end name='end' />
</workflow-app>
```

SIMPLE OOZIE EXAMPLE


```
<workflow-app name='wordcount-wf' xmlns="uri:oozie:workflow:0.1">
  <start to='wordcount' />
  <action name='wordcount'>
    <map-reduce>
      <job-tracker>${jobTracker}</job-tracker>
      <name-node>${nameNode}</name-node>
      <configuration>
        <property>
          <name>mapred.mapper.class</name>
          <value>org.myorg.WordCount.Map</value>
        </property>
        <property>
          <name>mapred.reducer.class</name>
```

- We specify what to do if the action ends successfully, and what to do if fails.
- In this example, if the job is successful, we go to the *end* node. Otherwise, we go to the *kill* node.

```
    <ok to='end' />
    <error to='kill' />
  </action>
  <kill name='kill'>
    <message>Something went wrong: ${wf:errorCode('wordcount')}</message>
  </kill>
  <end name='end' />
</workflow-app>
```

SIMPLE OOZIE EXAMPLE

```
<workflow-app name='wordcount-wf' xmlns="uri:oozie:workflow:0.1">
  <start to='wordcount' />
  <action name='wordcount'>
    <map-reduce>
      <job-tracker>${jobTracker}</job-tracker>
      <name-node>${nameNode}</name-node>
      <configuration>
        <property>
          <name>mapred.mapper.class</name>
          <value>org.myorg.WordCount.Map</value>
        </property>
        <property>
          <name>mapred.reducer.class</name>
          <value>org.myorg.WordCount.Reduce</value>
        </property>
      </configuration>
    </map-reduce>
  </action>
  <kill name='kill'>
    <message>Something went wrong: ${wf:errorCode('wordcount')}</message>
  </kill>
  <end name='end' />
</workflow-app>
```

- If the workflow reaches a *kill* node, it will kill all running actions and then terminate with an error.
- A workflow can have zero or more *kill* nodes.

```
<kill name='kill'>
  <message>Something went wrong: ${wf:errorCode('wordcount')}</message>
</kill>
<end name='end' />
</workflow-app>
```

SIMPLE OOZIE EXAMPLE

```

<workflow-app name='wordcount-wf' xmlns="uri:oozie:workflow:0.1">
  <start to='wordcount' />
  <action name='wordcount'>
    <map-reduce>
      <job-tracker>${jobTracker}</job-tracker>
      <name-node>${nameNode}</name-node>
      <configuration>
        <property>
          <name>mapred.mapper.class</name>
          <value>org.myorg.WordCount.Map</value>
        </property>
        <property>
          <name>mapred.reducer.class</name>
          <value>org.myorg.WordCount.Reduce</value>
        </property>
        <property>
          <name>mapred.input.dir</name>
          <value>${inputDir}</value>
        </property>
      </configuration>
    </map-reduce>
  </action>
  <end name='end' />
</workflow-app>

```

SIMPLE OOZIE EXAMPLE

- Every workflow must have an *end* node.
- This indicates that the workflow has completed successfully.

```

<end name='end' />
</workflow-app>

```

Other Oozie Control Nodes

A *decision* control node allows Oozie to determine the workflow execution path based on some criteria.

- Similar to a switch-case statement.

fork and *join* control nodes split one execution path into multiple execution paths which run concurrently.

- *fork* splits the execution path.
- *join* waits for all concurrent execution paths to complete before proceeding.
- *fork* and *join* are usually used in pairs.

Oozie Workflow Action Nodes

Node Name	Description
map-reduce	Runs either a Java MapReduce or Streaming job.
fs	Create directories, move or delete files or directories.
java	Runs the <i>main()</i> method in the specified Java class as a single-Map, Map-only job on the cluster.
pig	Runs a Pig script.
hive	Runs a Hive query.
sqoop	Runs a Sqoop job.
email	Sends an e-mail message.

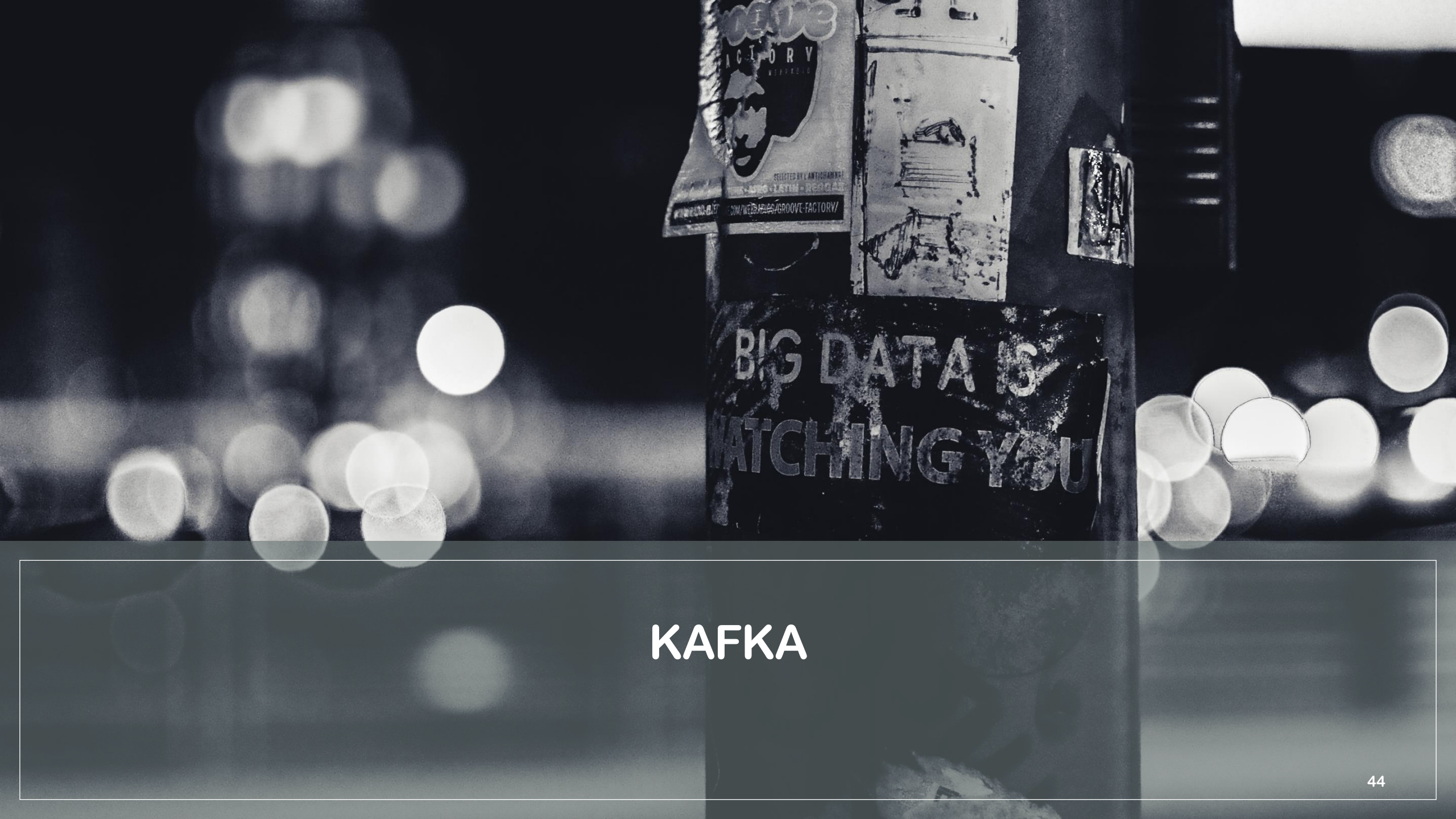
Submitting an Oozie Workflow

To submit an Oozie workflow using the command-line tool:

```
oozie job -oozie http://<oozie_server>/oozie -config config_file  
-run
```

Oozie can also be called from within a Java program

Via the Oozie client API.



KAFKA

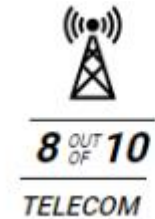
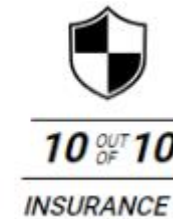
Apache Kafka

- Apache Kafka is an open-source distributed event streaming platform thousands of companies use for high-performance data pipelines, streaming analytics, data integration, and mission-critical applications.

- The image is a snapshot of the top-ten largest companies using Kafka, per-industry. The full list can be found at: <https://kafka.apache.org/powered-by>

APACHE KAFKA

More than **80% of all Fortune 100 companies** trust, and use Kafka.



10/10 Largest insurance companies

10/10 Largest manufacturing companies

10/10 Largest information technology and services companies

8/10 Largest telecommunications companies

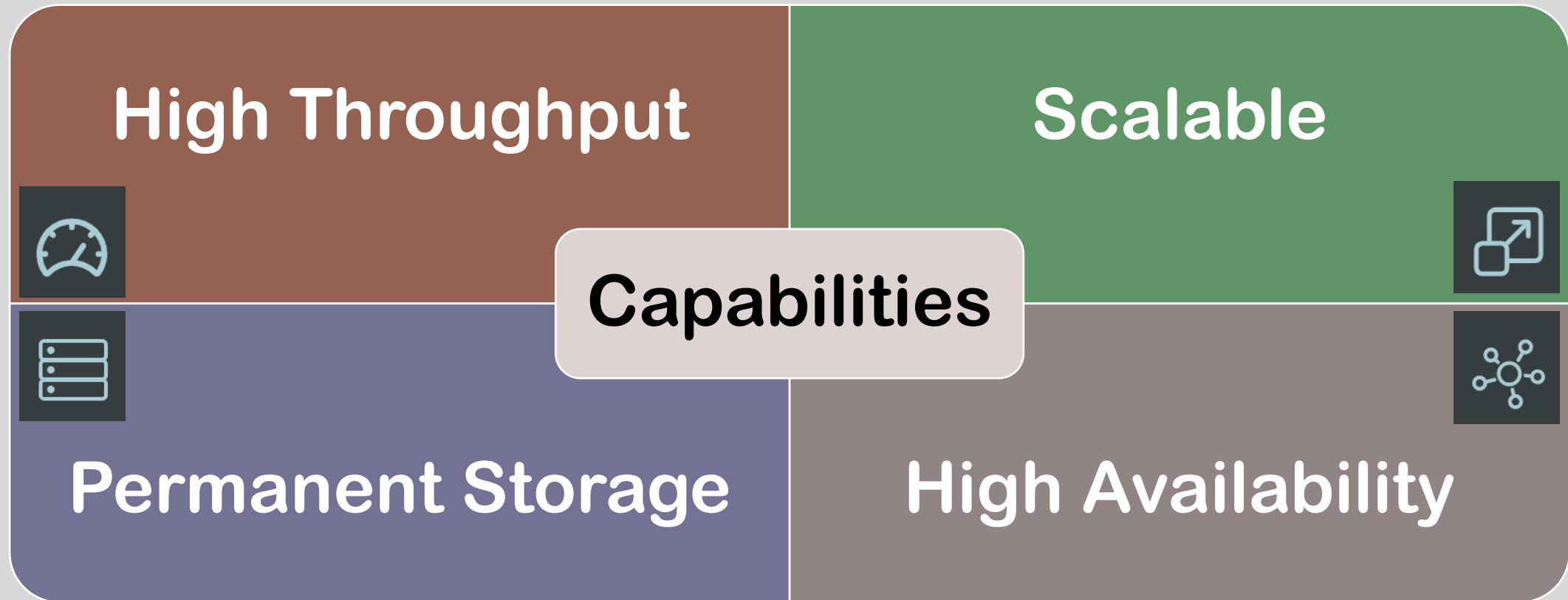
8/10 Largest transportation companies

7/10 Largest retail companies

7/10 Largest banks and finance companies

6/10 Largest energy and utilities organizations

Core Capabilities

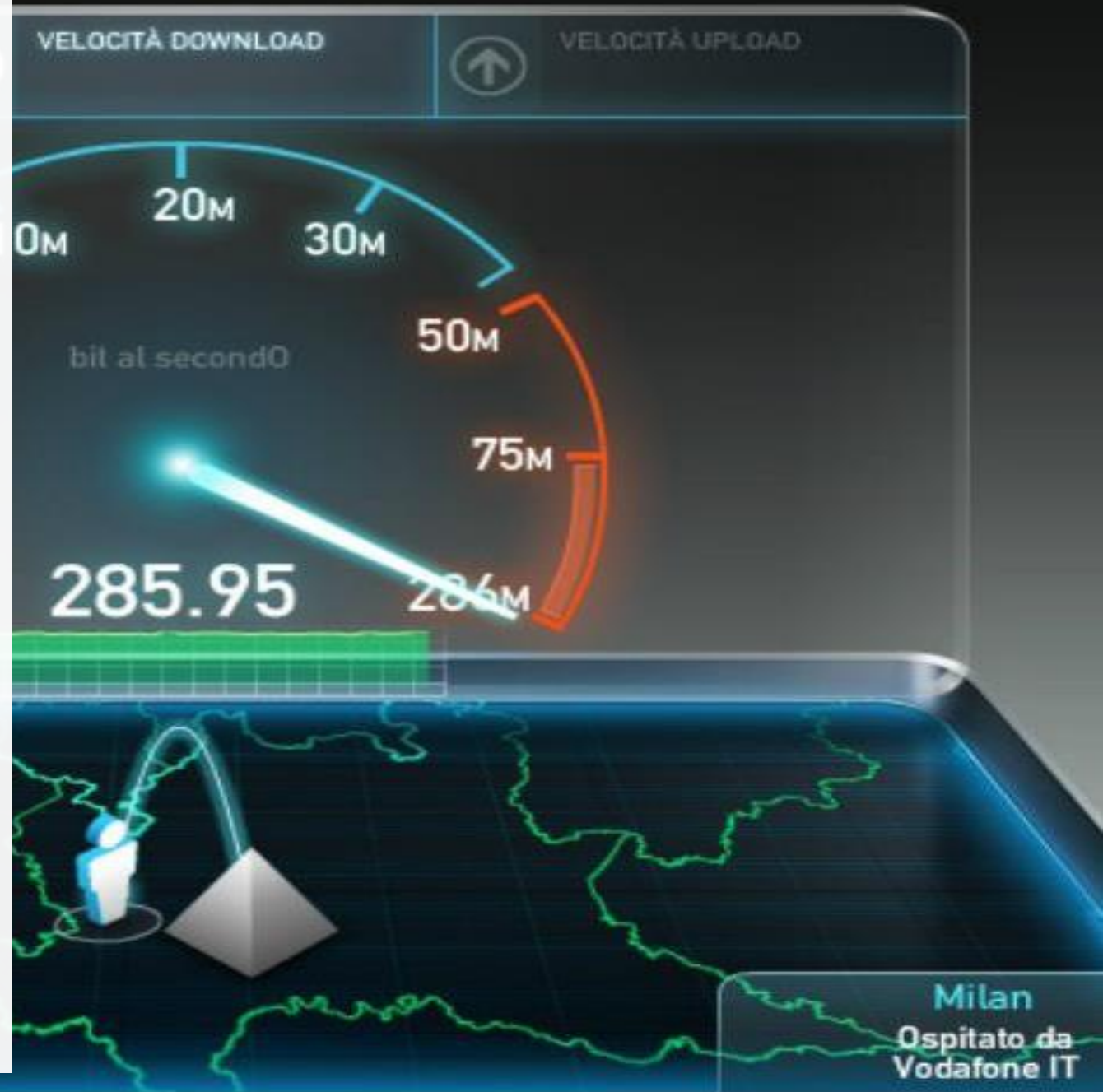


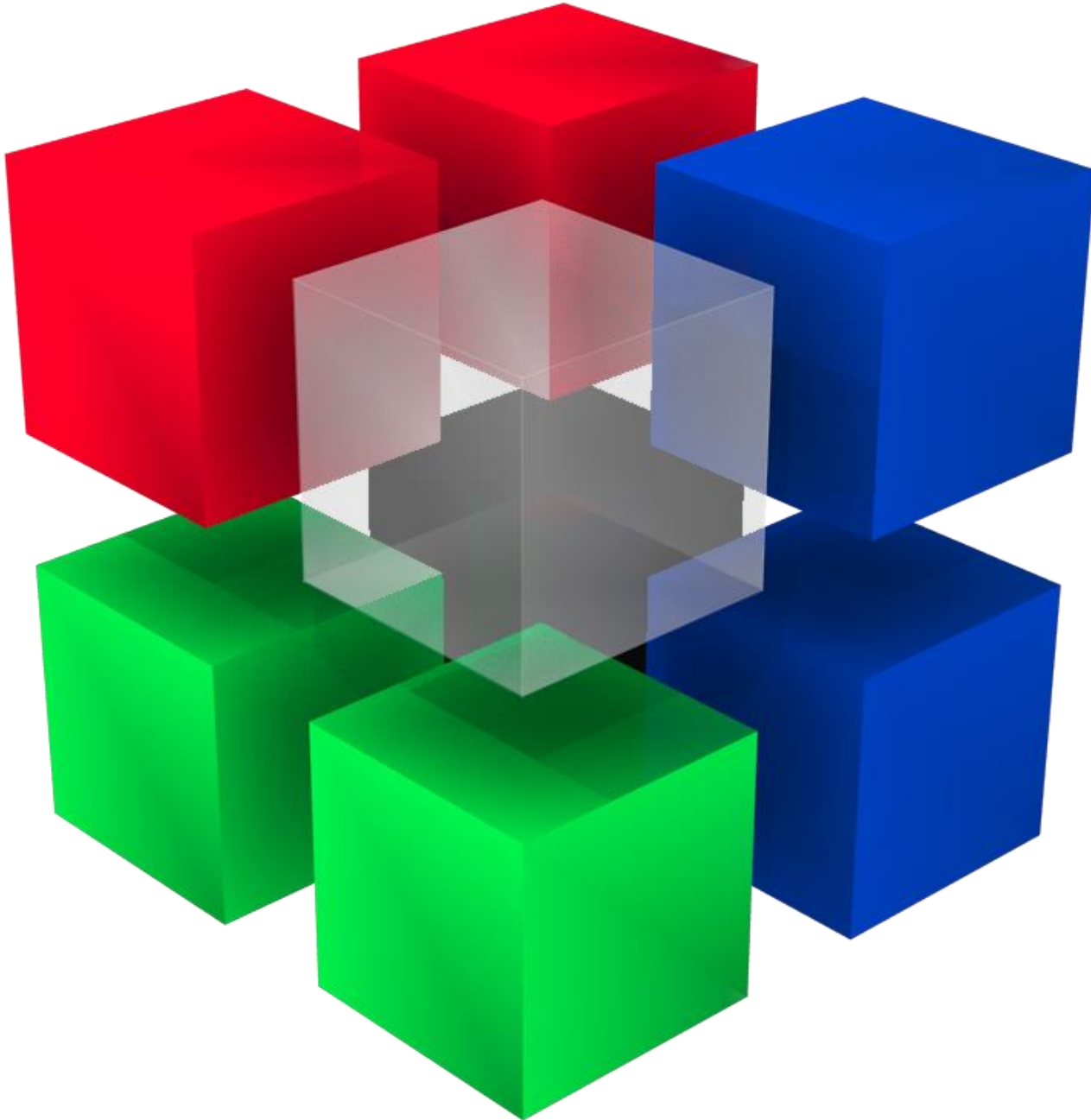
High Throughput

- Deliver messages at network-limited throughput using a cluster of machines with latencies as low as 2ms.



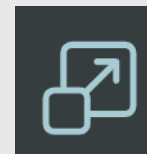
47





Scalable

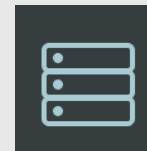
- Scale production clusters up to a thousand brokers, trillions of messages per day, petabytes of data, and hundreds of thousands of partitions. Elastically expand and contract storage and processing.





Permanent Storage

- Store streams of data safely in a distributed, durable, fault-tolerant cluster.



High Availability

- Stretch clusters efficiently over availability zones or connect separate clusters across geographic regions.



What is event streaming?

- Event streaming is the digital equivalent of the human body's central nervous system.
- It is the technological foundation for the 'always-on' world where businesses are increasingly software-defined and automated, and where the user of software is more software.
- Technically speaking, event streaming is the practice of capturing data in real-time from event sources like databases, sensors, mobile devices, cloud services, and software applications in the form of streams of events; storing these event streams durably for later retrieval; manipulating, processing, and reacting to the event streams in real-time as well as retrospectively, and routing the event streams to different destination technologies as needed.
- Event streaming thus ensures a continuous flow and interpretation of data so that the right information is at the right place and time.

What Can I Use Event Streaming for?

- Event streaming is applied to a wide variety of use cases across many industries and organisations. Its many examples include:

•To process payments and financial transactions in real-time, such as in stock exchanges, banks, and insurances.

Finance



•To track and monitor cars, trucks, fleets, and shipments in real-time, such as in logistics and the automotive industry.

Tracking



•To continuously capture and analyse sensor data from IoT devices or other equipment, such as in factories and wind parks.

IoT



•To collect and immediately react to customer interactions and orders, such as in retail, the hotel and travel industry, and mobile applications.

Transactions



•To monitor patients in hospital care and predict changes in condition to ensure timely treatment in emergencies.

Medical



•To connect, store, and make available data produced by different divisions of a company.

Departments



•To serve as the foundation for data platforms, event-driven architectures, and microservices.

Platform



Apache Kafka is an Event Streaming Platform

- Kafka combines three key capabilities so you can implement your use cases for event streaming end-to-end with a single battle-tested solution:
 1. To **publish** (write) and **subscribe to** (read) streams of events, including continuous import/export of your data from other systems.
 2. To **store** streams of events durably and reliably for as long as you want.
 3. To **process** streams of events as they occur or retrospectively.
- And all this functionality is provided in a distributed, highly scalable, elastic, fault-tolerant, and secure manner.
- Kafka can be deployed on bare-metal hardware, virtual machines, and containers, and on-premises as well as in the cloud.
- You can choose between self-managing your Kafka environments and using fully managed services offered by various vendors.

How Does Kafka Work in a Nutshell?

- Kafka is a distributed system consisting of **servers** and **clients** communicating via a high-performance TCP network protocol. It can be deployed on bare-metal hardware, virtual machines, and containers in on-premise and cloud environments.
- **Servers:** Kafka is run as a cluster of one or more servers that can span multiple data centres or cloud regions. Some of these servers form the storage layer, called the brokers. Other servers run [Kafka Connect](#) to continuously import and export data as event streams to integrate Kafka with your existing systems, such as relational databases as well as other Kafka clusters. To let you implement mission-critical use cases, a Kafka cluster is highly scalable and fault-tolerant: if any of its servers fails, the other servers will take over their work to ensure continuous operations without any data loss.
- **Clients:** They allow you to write distributed applications and microservices that read, write, and process streams of events in parallel, at scale, and in a fault-tolerant manner, even in the case of network problems or machine failures. Kafka ships with some such clients included, which are augmented by [dozens of clients](#) provided by the Kafka community: clients are available for Java and Scala, including the higher-level [Kafka Streams](#) library, for Go, Python, C/C++, and many other programming languages, as well as REST APIs.

Main Concepts and Terminology

- An **event** records the fact that "something happened" in the world or your business. It is also called a record or message in the documentation. When you read or write data to Kafka, you do this in the form of events. Conceptually, an event has a key, value, timestamp, and optional metadata headers. Here's an example event:
 - Event key: "Alice"
 - Event value: "Made a payment of \$200 to Bob"
 - Event timestamp: "Jun. 25, 2023 at 2:06 p.m."
- **Producers** are those client applications that publish (write) events to Kafka, and **consumers** are those that subscribe to (read and process) these events.
- In Kafka, producers and consumers are fully decoupled and agnostic of each other, which is a key design element to achieve the high scalability that Kafka is known for. For example, producers never need to wait for consumers. Kafka provides various guarantees such as the ability to process events exactly once.

Main Concepts and Terminology (2)

- Events are organised and durably stored in **topics**. Very simplified, a topic is similar to a folder in a filesystem, and the events are the files in that folder. An example topic name could be "payments". Topics in Kafka are always multi-producer and multi-subscriber: a topic can have zero, one, or many producers that write events to it, as well as zero, one, or many consumers that subscribe to these events. Events in a topic can be read as often as needed—unlike traditional messaging systems, events are not deleted after consumption. Instead, you define for how long Kafka should retain your events through a per-topic configuration setting, after which old events will be discarded. Kafka's performance is effectively constant with respect to data size, so storing data for a long time is perfectly fine.
- Topics are **partitioned**, meaning a topic is spread over a number of "buckets" located on different Kafka brokers. This distributed placement of your data is very important for scalability because it allows client applications to both read and write the data from/to many brokers at the same time. When a new event is published to a topic, it is actually appended to one of the topic's partitions. Events with the same event key (e.g., a customer or vehicle ID) are written to the same partition, and Kafka guarantees that any consumer of a given topic-partition will always read that partition's events in exactly the same order as they were written.

Main Concepts and Terminology (3)

To make your data fault-tolerant and highly available, every topic can be **replicated**, even across geo-regions or data centres, so that there are always multiple brokers that have a copy of the data just in case things go wrong, you want to do maintenance on the brokers, and so on.

A common production setting is a replication factor of 3, i.e., there will always be three copies of your data.

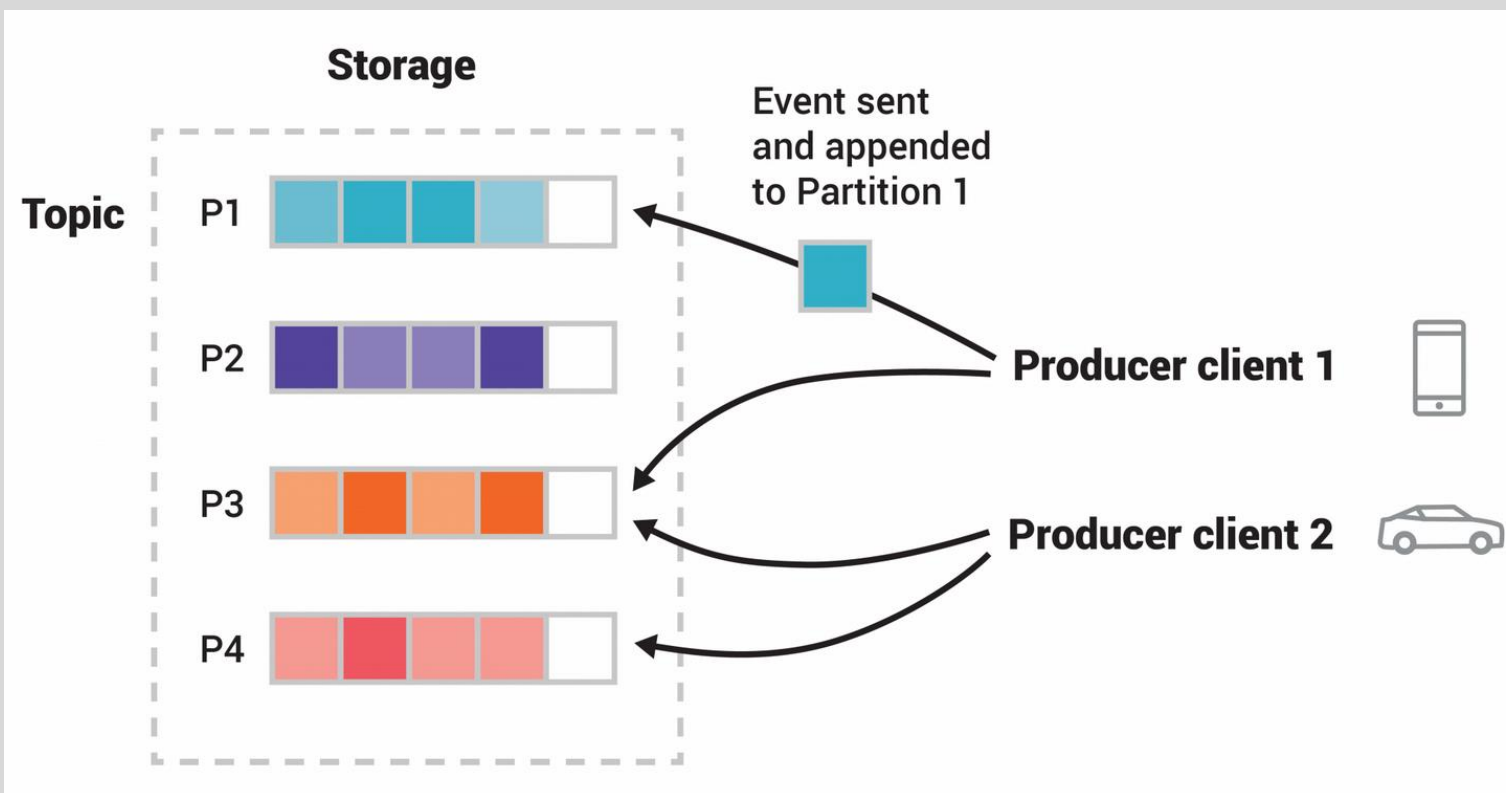
This replication is performed at the level of topic-partitions.

This primer should be sufficient for an introduction.

The [Design](#) section of the documentation explains Kafka's various concepts in full detail if you are interested.

An Example Topic in Kafka

- This example topic has four partitions (P1 – P4). Two different **producer clients** are publishing, independently from each other, new events to the topic by writing events over the network to the topic's partitions. Events with the same key (denoted by their colour in the figure) are written to the same partition.



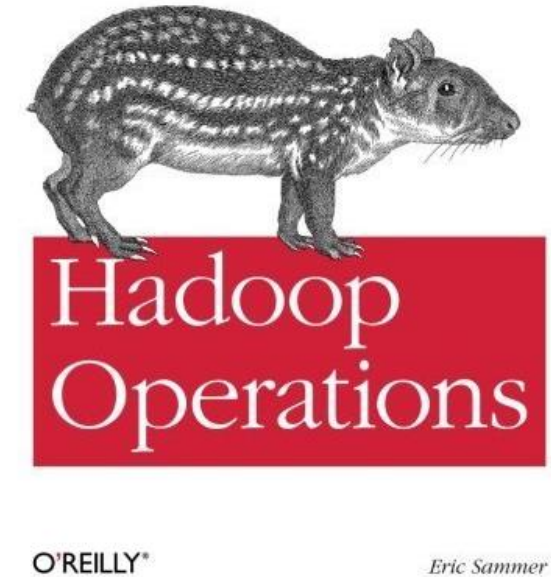
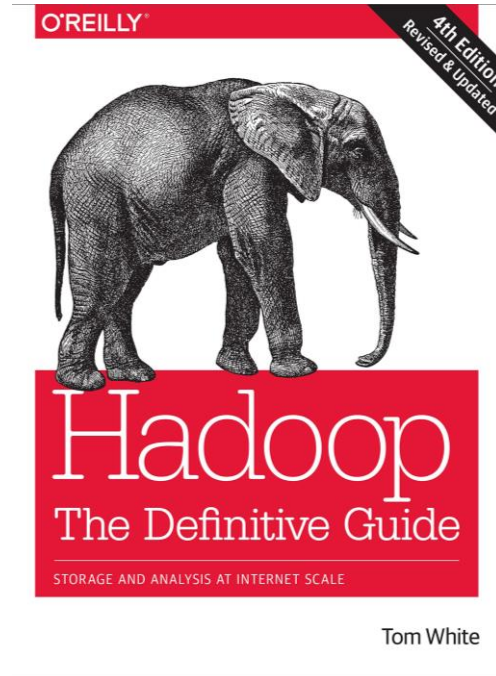
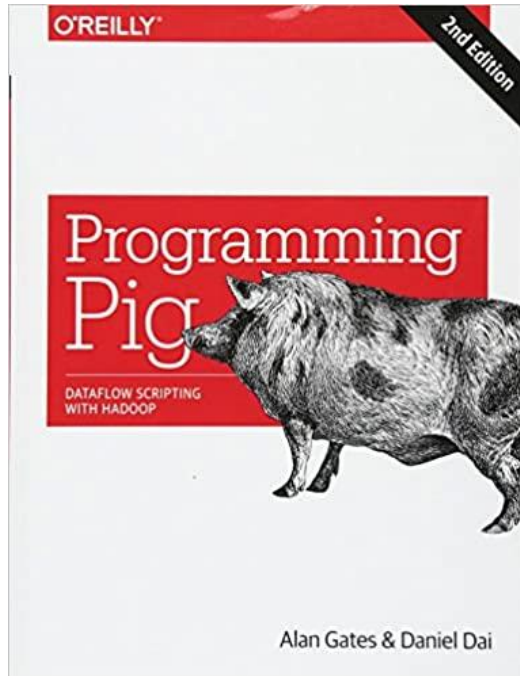
Kafka APIs

In addition to command line tooling for management and administration tasks, Kafka has five core APIs for Java and Scala:

- The [Admin API](#) to manage and inspect topics, brokers, and other Kafka objects.
- The [Producer API](#) to publish (write) a stream of events to one or more Kafka topics.
- The [Consumer API](#) to subscribe to (read) one or more topics and to process the stream of events produced to them.
- The [Kafka Streams API](#) to implement stream processing applications and microservices. It provides higher-level functions to process event streams, including transformations, stateful operations like aggregations and joins, windowing, processing based on event-time, and more. Input is read from one or more topics in order to generate output to one or more topics, effectively transforming the input streams to output streams.
- The [Kafka Connect API](#) to build and run reusable data import/export connectors that consume (read) or produce (write) streams of events from and to external systems and applications so they can integrate with Kafka. For example, a connector to a relational database like PostgreSQL might capture every change to a set of tables. However, in practice, you typically don't need to implement your own connectors because the Kafka community already provides hundreds of ready-to-use connectors.

Short Video Introduction to Kafka





Texts and Resources

- Unless stated otherwise, the materials presented in this lecture are taken from:
 - Hadoop: the definitive guide, White, Tom (Tom E.) author., 4th ed., Sebastopol, California: O'Reilly, 2015.
 - Hadoop operations, Sammer, Eric., Loukides, Michael Kosta.; Nash, Courtney.; Romano, Robert (Illustrator), illustrator., First edition., Sebastopol, CA: O'Reilly, 2012.
 - Programming pig: dataflow scripting with hadoop, Gates, Alan, author.; Dai, Daniel, 2nd edition., Beijing, China: O'Reilly, 2017



Unit Summary

- Data Acquisition.
- Introduction to Kafka.

Summary

